

Modul mahasiswa - Lab 02: Linux, berkas, izin, dan Bash

COMP6991031 · sesi 2. Semua perubahan dilakukan pada data latihan di `workspace/`. Baca [README lab](#) dan [panduan Git](#). Gambar di bawah menunjukkan keluaran **salinan uji**, bukan bukti tugas Anda.

Tujuan dan teori ringkas

Anda akan memakai path absolut/relatif, membaca struktur filesystem Linux (`/`, `/home`, `/etc`, `/var`, `/tmp`), mengolah berkas dan log, memakai pipe `|` serta redirection `>/>>`, membaca izin `rwx`, dan menyelesaikan skrip Bash dengan parameter, percabangan, serta exit code. Output perintah menentukan apa yang terjadi pada proses berikutnya; karena itu file hasil dan exit code sama pentingnya dengan teks yang tercetak.

Prasyarat

- Terminal **Bash**: WSL Ubuntu, Linux, macOS dengan Bash, GitHub Codespaces, atau Git Bash di Windows. Perintah `.sh` di modul ini dijalankan dari Bash, **bukan PowerShell biasa**. Git Bash berhasil pada uji paket.
- Repo pribadi Lab 02, editor teks, dan Git untuk push. Lab ini tidak membutuhkan Docker, akun cloud, `sudo`, atau internet setelah repo tersedia.
- Pastikan terminal dimulai dari **root repo**. Path pada contoh keluaran dapat berbeda antar mesin.

1. Buat workspace dan baca hasil awal

```
pwd
bash scripts/setup.sh
bash scripts/inspect.sh
cd workspace
pwd
ls -la
```

`setup.sh` hanya membuat `workspace/` bila belum ada; menjalankannya ulang **tidak menghapus** pekerjaan.

`inspect.sh` hanya membaca data. Gambar ini berasal dari perintah tersebut pada salinan uji; path `/tmp/...` hanya path mesin penguji.

```

Lab 02: membuat dan memeriksa workspace — keluaran aktual dari salinan uji

$ bash scripts/setup.sh; bash scripts/inspect.sh
Workspace siap: /tmp/lab02_capture_rybeqlwd/lab02/workspace
Jalankan: cd "/tmp/lab02_capture_rybeqlwd/lab02/workspace"

[1] Direktori kerja dan path absolut
/tmp/lab02_capture_rybeqlwd/lab02
/tmp/lab02_capture_rybeqlwd/lab02/workspace

[2] Daftar isi dan izin
total 8
drwxr-xr-x 1 Lenovo 197121 0 Oct 1 01:22 .
drwxr-xr-x 1 Lenovo 197121 0 Oct 1 01:22 ..
drwxr-xr-x 1 Lenovo 197121 0 Oct 1 01:22 backup
drwxr-xr-x 1 Lenovo 197121 0 Oct 1 01:22 docs
drwxr-xr-x 1 Lenovo 197121 0 Oct 1 01:22 logs
drwxr-xr-x 1 Lenovo 197121 0 Oct 1 01:22 results
drwxr-xr-x 1 Lenovo 197121 0 Oct 1 01:22 scripts
drwxr-xr-x 1 Lenovo 197121 0 Oct 1 01:22 tmp

[3] Dua baris pertama CSV
method,path,status
GET,/health,200

[4] Dua baris terakhir log
2026-09-30T08:03:00Z WARN GET /retry 429
2026-09-30T08:04:00Z ERROR GET /missing 404

[5] Pipeline: jumlah baris WARN
2

Skrrip ini hanya membaca file; lanjutkan tugas manual di README.

```

Keluaran nyata setup dan inspect Lab 02

- **Langkah:** Jalankan `bash scripts/setup.sh` lalu `bash scripts/inspect.sh` dari root repo. **Fungsi:** Membuat workspace latihan dan memeriksa berkas awal tanpa mengubahnya lagi. **Cara kerja:** Skrip setup menyalin struktur contoh; inspect membaca CSV, log, direktori, dan permission. **Baca hasil:** Cari folder `docs/logs/backup/results/scripts/tmp`, dua baris CSV, akhir log, serta hitungan `WARN 2`.

Checkpoint 1: ada folder `docs`, `logs`, `backup`, `results`, `scripts`, `tmp`. Dua baris pertama CSV, dua baris akhir log, dan jumlah `WARN 2` terlihat.

2. Navigasi dan baca berkas

Dari `workspace/`, jalankan:

```

pwd
cd docs
pwd
cd ..
ls -la docs
find . -type f | sort
cat docs/readme.txt
head -n 2 docs/requests.csv
tail -n 2 logs/app.log
grep 'ERROR' logs/app.log
less logs/app.log

```

Tekan `q` untuk keluar dari `less`. Jika `tree` tersedia, bandingkan `tree .` dengan `find`. **Checkpoint 2:** Anda dapat menyebutkan kapan `docs/readme.txt` adalah path relatif yang benar, apa arti `.` dan `..`, dan mengapa `ls -l` memperlihatkan kelompok izin pemilik/grup/lainnya.

Git Bash · labs/02/workspace

```
$ pwd
/d/cloud_services/.build/audit_lab02_20261001/workspace

$ ls -l docs/requests.csv logs/app.log scripts/check-status.sh
-rw-r--r-- 1 Lenovo 197121 83 Oct 1 07:21 docs/requests.csv
-rw-r--r-- 1 Lenovo 197121 211 Oct 1 07:21 logs/app.log
-rwxr-xr-x 1 Lenovo 197121 442 Oct 1 07:21 scripts/check-status.sh

$ head -n 2 docs/requests.csv
method,path,status
GET,/health,200

$ tail -n 2 logs/app.log
2026-09-30T08:03:00Z WARN GET /retry 429
2026-09-30T08:04:00Z ERROR GET /missing 404

$ grep ERROR logs/app.log
2026-09-30T08:02:00Z ERROR POST /orders 500
2026-09-30T08:04:00Z ERROR GET /missing 404
```

Perintah navigasi dan pembacaan CSV serta log pada Git Bash

- **Langkah:** Dari `workspace`, jalankan `pwd`, `ls -la`, `head -n 2 docs/requests.csv`, `tail -n 2 logs/app.log`, dan `grep 'ERROR' logs/app.log`. **Fungsi:** Melatih path, permission, dan pembacaan sebagian berkas. **Cara kerja:** Shell menghitung path dari working directory; head/tail memilih baris dan grep menyaring teks. **Baca hasil:** Baca path aktif, tiga kelompok izin, header/data CSV, dan baris log yang cocok.

Ikuti urutan `pwd`, `ls -l`, `head`, `tail`, lalu `grep`. Path absolut, jam, dan nama pengguna di komputer Anda akan berbeda.

3. Kerjakan artefak berkas dan log

Kerjakan tugas berikut **sendiri** dari `workspace/`; gunakan petunjuk perintah pada kolom tengah. Jangan mengedit `logs/app.log` asal.

Tugas	Perintah yang perlu dipakai	Bukti akhir	
Salin dokumen	<code>cp</code>	<code>backup/readme-copy.txt</code> identik dengan <code>docs/readme.txt</code>	
Ubah nama draf	<code>mv</code>	<code>docs/notes.txt</code> ada, <code>docs/draft.txt</code> hilang	
Buat penanda	<code>touch</code>	<code>results/visited.txt</code> ada	
Saring error	<code>grep</code> lalu <code>></code>	<code>results/errors.txt</code> berisi dua baris ERROR	
Hitung warning	<code>grep</code> , <code>`</code>	<code>, wc -l, >`</code>	<code>results/warn-count.txt</code> berisi 2

Untuk melihat beda `>` dan `>>` tanpa merusak log:

```
tail -n 1 logs/app.log > results/incident.txt
printf 'Analisis: perlu cek endpoint.\n' >> results/incident.txt
cat results/incident.txt
```

Checkpoint 3: Anda dapat menjelaskan mengapa `>` menimpa file tujuan sementara `>>` menambah baris. Jangan gunakan `rm -rf`; bila berlatih hapus, gunakan `rm -i tmp/remove-me.txt` pada file latihan saja.

Git Bash · labs/02/workspace

```
$ find backup docs results -maxdepth 2 -type f | sort
backup/readme-copy.txt
docs/notes.txt
docs/readme.txt
docs/requests.csv
results/errors.txt
results/incident.txt
results/visited.txt
results/warn-count.txt

$ cat results/errors.txt
2026-09-30T08:02:00Z ERROR POST /orders 500
2026-09-30T08:04:00Z ERROR GET /missing 404

$ cat results/warn-count.txt
2

$ cat results/incident.txt
2026-09-30T08:04:00Z ERROR GET /missing 404
Analisis: perlu cek endpoint.
```

Artefak file hasil `cp`, `mv`, `grep`, `pipeline`, dan `redirection`

- **Langkah:** Jalankan tugas `cp`, `mv`, `grep`, `pipeline` `|`, serta `redirection` `>/>>` pada folder `workspace`. **Fungsi:** Membuat artefak backup dan ringkasan insiden yang akan diverifikasi. **Cara kerja:** Perintah menyalin/memindah berkas, menyaring log, menghitung baris, lalu menulis hasil ke `results/`. **Baca hasil:** Periksa nama file tujuan, dua baris `ERROR`, hitungan `WARN` 2, dan isi `incident.txt`.

Periksa nama file di `backup/`, `docs/`, dan `results/`, lalu cocokkan dua baris `ERROR`, angka 2, dan isi `incident.txt`. Gambar diambil setelah tugas dikerjakan pada salinan uji; buat hasil sendiri sebelum menjalankan verifier.

4. Lengkapi skrip HTTP dan verifikasi

Edit `workspace/scripts/check-status.sh` yang masih berisi `TODO`. Spesifikasinya:

Input	Teks keluaran persis	Exit code
100-199	INFO: HTTP <code>	0
200-299	OK: HTTP <code>	0
300-399	REDIRECT: HTTP <code>	0
400-599	ALERT: HTTP <code>	0
Kosong, bukan tiga digit, atau di luar 100-599	Usage: check-status.sh <HTTP-code> ke stderr	2

Pakai parameter pertama melalui `code="${1:-}"`, validasi input, dan `if/elif/else`. Setelah menyimpan:

```

chmod u+x scripts/check-status.sh
./scripts/check-status.sh 103
./scripts/check-status.sh 200
./scripts/check-status.sh 302
./scripts/check-status.sh 503
./scripts/check-status.sh
echo "$?"
cd ..
bash scripts/verify.sh

```

Checkpoint 4: empat kode valid menghasilkan kategori yang sesuai; panggilan kosong memberi exit code 2. Verifier berakhir **Hasil: 0 gagal**. Gambar berikut adalah hasil **setelah solusi dikerjakan pada salinan uji**; starter mahasiswa memang belum demikian.

```

Lab 02 - skrip status dan semua keluaran

$ nl -ba scripts/check-status.sh | head -n 22
1  #!/usr/bin/env bash
2  set -euo pipefail
3  code="${1:-}"
4  if [[ ! "$code" =~ ^[1-5][0-9][0-9]$ ]]; then
5    printf 'Usage: check-status.sh <HTTP-code>\n' >&2
6    exit 2
7  elif (( code < 200 )); then
8    printf 'INFO: HTTP %s\n' "$code"
9  elif (( code < 300 )); then
10   printf 'OK: HTTP %s\n' "$code"
11 elif (( code < 400 )); then
12   printf 'REDIRECT: HTTP %s\n' "$code"
13 else
14   printf 'ALERT: HTTP %s\n' "$code"
15 fi

$ for c in 103 200 302 503 99; do ./scripts/check-status.sh "$c"; printf 'exit:%s\n' "$?"; done
INFO: HTTP 103
exit=0
OK: HTTP 200
exit=0
REDIRECT: HTTP 302
exit=0
ALERT: HTTP 503
exit=0
exit=2

Usage: check-status.sh <HTTP-code>

$ bash scripts/verify.sh
OK cp membuat salinan identik
OK mv mengubah draft.txt menjadi notes.txt
OK touch membuat visited.txt
OK grep dan redirection menghasilkan errors.txt
OK pipeline menghitung WARN
OK izin eksekusi check-status.sh aktif
OK HTTP 103 -> INFO
OK HTTP 200 -> OK
OK HTTP 302 -> REDIRECT
OK HTTP 503 -> ALERT
OK input kosong ditolak

Hasil: 0 gagal

Output aktual dan salinan terisolasi setelah kunci diterapkan, repo template tetap berisi TODO.

```

Perintah: `nl -ba scripts/check-status.sh`, jalankan kode 103/200/302/503/99, lalu `bash scripts/verify.sh`. Fungsi: menguji seluruh cabang if/elif/else, input tak valid, dan artefak. Cara kerja: regex menerima 100-599; verifier memeriksa file dan keluaran skrip. Baca hasil: kategori sesuai, input 99 exit=2, dan Hasil: 0 gagal. Ini render output command aktual dari salinan terisolasi; starter repo tetap berisi TODO.

Lab 02: hasil setelah tugas dikerjakan — keluaran aktual dari salinan uji

```
$ bash scripts/verify.sh
OK  cp membuat salinan identik
OK  mv mengubah draft.txt menjadi notes.txt
OK  touch membuat visited.txt
OK  grep dan redirection menghasilkan errors.txt
OK  pipeline menghitung WARN
OK  izin eksekusi check-status.sh aktif
OK  HTTP 103 -> INFO
OK  HTTP 200 -> OK
OK  HTTP 302 -> REDIRECT
OK  HTTP 503 -> ALERT
OK  input kosong ditolak
```

Hasil: 0 gagal

Hasil nyata verify Lab 02 setelah tugas selesai

- **Langkah:** Setelah tugas berkas dan `check-status.sh` selesai, jalankan `bash scripts/verify.sh` dari root repo.
Fungsi: Memeriksa artefak dan klasifikasi status HTTP secara berulang. **Cara kerja:** Verifier membaca file hasil, menjalankan skrip status, dan menghitung pemeriksaan gagal. **Baca hasil:** Baris akhir harus `Hasil: 0 gagal`; kegagalan starter sebelum tugas selesai adalah petunjuk kerja.

Pertanyaan yang dijawab di laporan

1. Tulis satu contoh path absolut dan relatif menuju file yang sama. Dari direktori mana masing-masing berlaku?
2. Apa perbedaan `|`, `>`, dan `>>` pada hasil log ini?
3. Mengapa `chmod u+x` diperlukan untuk `./scripts/check-status.sh`, tetapi `bash scripts/check-status.sh 200` masih dapat dijalankan tanpa izin eksekusi?
4. Apa arti `rwX` untuk pemilik, grup, dan pengguna lain? Kapan `sudo` atau `chown` memang diperlukan? Jangan menjalankannya untuk lab ini.
5. Mengapa kode HTTP `103` masuk kategori `INFO` dan `503` masuk `ALERT`, tetapi exit code skrip tetap `0` untuk keduanya?

Bukti, commit, dan push

Salin [hasil/TEMPLATE_LAPORAN.md](#) menjadi `hasil/lab02.md` dari root repo. Isi jawaban, simpan screenshot terminal **Anda sendiri** (`pwd`, `ls -la`, `bash scripts/verify.sh`) di `hasil/bukti/`, dan catat lingkungan Bash yang digunakan. File `workspace/results/` serta `workspace/scripts/check-status.sh` adalah artefak yang diperiksa.

Kembali ke root repo, lalu:

```
git status --short
git add scripts starter workspace hasil/lab02.md hasil/bukti
git diff --cached --name-only
git diff --cached --check
git diff --cached
git commit -m "lab02: operasi Linux dan skrip status HTTP"
git push
```

Lab 02 · cek repo Git terbit

```
$ git remote -v
origin https://github.com/SeedFlora/meet2CloudService.git (fetch)
origin https://github.com/SeedFlora/meet2CloudService.git (push)
```

```
$ git status --short
?? screenshots/lab02_kode_status_langkah4.png
```

```
$ git log -1 --oneline
fb51088 Publish complete Lab 02 modules
```

```
$ git rev-parse HEAD; git ls-remote origin refs/heads/main
local: fb51088b9bd7029cc90576965be233f6a299b61a
remote: fb51088b9bd7029cc90576965be233f6a299b61a
Sama: True
```

Output aktual repo template SeedFlora. Mahasiswa melakukan commit/push pada repo pribadi.

SHA pada gambar adalah snapshot saat uji. Setelah modul diperbarui, jalankan ulang perintah untuk memeriksa commit terbaru.

Perintah: `git remote -v`, `git status --short`, `git log -1 --oneline`, `git rev-parse HEAD`, dan `git ls-remote origin refs/heads/main`. Fungsi: memastikan origin benar dan commit terbaru ada di GitHub. Cara kerja: bandingkan SHA lokal dengan SHA remote. Baca hasil: Sama: True pada repo template pengajar; ulangi pada repo pribadi sesudah push. Ini render output command aktual.

Pastikan berkas staged hanya berisi data latihan. Jangan push file pribadi, token, `.env`, atau screenshot yang memperlihatkan kredensial. Bila push pertama memerlukan upstream, jalankan `git push -u origin main` bila branch Anda `main`.

Berhenti dan mengatasi masalah

Tidak ada service yang perlu dimatikan. Jika `tree` tidak ada, pakai `find`. Jika `./scripts/check-status.sh` memberi `Permission denied`, ulangi `chmod u+x` atau jalankan dengan `bash`. Jika verifier gagal, baca baris `FAIL` satu per satu; `setup.sh` tidak akan mengatur ulang workspace. Jika Anda berada di folder yang salah, gunakan `pwd` lalu `cd` ke root repo sebelum `bash scripts/verify.sh`.

Kunci praktik lengkap dan cara membacanya

Kerjakan dari **root repo Lab 02** di Bash. Kunci ini sengaja ditempatkan di modul mahasiswa agar Anda dapat membandingkan setiap artefak dengan hasil sendiri. Sebelum menyalin jawaban, coba dulu tabel tugas pada bagian 3.

A. Operasi berkas dan log

```
bash scripts/setup.sh
cd workspace
cp docs/readme.txt backup/readme-copy.txt
mv docs/draft.txt docs/notes.txt
touch results/visited.txt
grep 'ERROR' logs/app.log > results/errors.txt
grep 'WARN' logs/app.log | wc -l > results/warn-count.txt
cat results/errors.txt
cat results/warn-count.txt
```

`cp` menggandakan isi sambil menyisakan sumber. `mv` mengganti nama file. `touch` membuat file kosong. `grep` memilih baris log; `>` menyimpan hasil ke file. Pada hitungan `WARN`, pipe `|` memberi keluaran `grep` ke `wc -l` tanpa file perantara. Pada data awal, `errors.txt` berisi dua baris dan `warn-count.txt` berisi 2.

B. Seluruh isi check-status.sh

Bila editor sulit dipakai, dari `workspace/` salin seluruh perintah berikut ke terminal Bash. `cat > ... <<'EOF'` mengganti isi file tujuan; tanda kutip pada `EOF` menjaga `$code` agar tidak dievaluasi saat file ditulis. Jangan ubah `scripts/verify.sh`, karena itulah pemeriksa pekerjaan Anda.

```
cat > scripts/check-status.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail

code="${1:-}"
if [[ ! "$code" =~ ^[1-5][0-9][0-9]$ ]]; then
    printf 'Usage: check-status.sh <HTTP-code>\n' >&2
    exit 2
elif (( code < 200 )); then
    printf 'INFO: HTTP %s\n' "$code"
elif (( code < 300 )); then
    printf 'OK: HTTP %s\n' "$code"
elif (( code < 400 )); then
    printf 'REDIRECT: HTTP %s\n' "$code"
else
    printf 'ALERT: HTTP %s\n' "$code"
fi
EOF
```

Regex memastikan tepat tiga digit dari 100 sampai 599. `${1:-}` memberi string kosong ketika parameter tidak diberikan. `>&2` mengirim kesalahan penggunaan ke `stderr`; `exit 2` menandai input tidak valid. Status HTTP 503 tetap menghasilkan exit code skrip `0` karena skrip berhasil *mengklasifikasikan* input tersebut. Gunakan kode keluar 2 hanya untuk argumen yang salah.

C. Verifikasi dan hasil

```
chmod u+x scripts/check-status.sh
./scripts/check-status.sh 103
./scripts/check-status.sh 200
./scripts/check-status.sh 302
./scripts/check-status.sh 503
./scripts/check-status.sh 99
echo "$?"
cd ..
bash scripts/verify.sh
```

Empat input pertama masing-masing memberi INFO, OK, REDIRECT, ALERT. Input 99 memberi pesan Usage dan echo "\$?" menunjukkan 2. Verifier akhir harus menunjukkan Hasil: 0 gagal. Setelah itu salin hasil/TEMPLATE_LAPORAN.md menjadi hasil/lab02.md, simpan bukti sendiri di hasil/bukti/, lalu ikuti [panduan Git](#).

```
Lab 02 - Git Bash, salinan kerja terisolasi
bash scripts/setup.sh -> workspace dibuat
bash scripts/verify.sh (starter) -> gagal sesuai tugas

Kunci: cp, mv, touch, grep, pipe, chmod, Bash if/elif/else
OK  cp membuat salinan identik
OK  mv mengubah draft.txt menjadi notes.txt
OK  touch membuat visited.txt
OK  grep dan redirection menghasilkan errors.txt
OK  pipeline menghitung WARN
OK  izin eksekusi check-status.sh aktif
OK  HTTP 103 -> INFO
OK  HTTP 200 -> OK
OK  HTTP 302 -> REDIRECT
OK  HTTP 503 -> ALERT
OK  input kosong ditolak
Hasil: 0 gagal

bash workspace/scripts/check-status.sh 99 -> exit 2
```

Output uji aktual ditata ulang agar terbaca; pekerjaan mahasiswa dilakukan di repo sendiri.

Perintah: `bash scripts/setup.sh`, lima operasi berkas, isi skrip status, `chmod u+x`, lalu `bash scripts/verify.sh`. Gambar adalah transkrip uji aktual yang ditata agar terbaca. Starter memberi 10 kegagalan sesuai tugas; setelah kunci diterapkan pada salinan terisolasi, semua baris OK dan Hasil: 0 gagal. Input 99 diuji terpisah dan keluar dengan kode 2.

D. Kunci pertanyaan diskusi

1. Dari root repo, path relatif `workspace/logs/app.log` menunjuk file yang sama dengan path absolut hasil `realpath workspace/logs/app.log`. Jika direktori aktif pindah ke `workspace/`, path relatifnya menjadi `logs/app.log`.
2. | menghubungkan stdout perintah kiri ke stdin perintah kanan. > membuat atau mengganti isi file tujuan. >> menambahkan di akhir file tujuan.
3. Izin x membuat file bisa dieksekusi langsung dengan `./...; bash file.sh` meminta Bash membaca file sehingga cukup izin baca.
4. `rwX` berarti baca, tulis, eksekusi untuk masing-masing kelompok pemilik, grup, dan lainnya. `sudo` dipakai hanya ketika operasi yang sah memang memerlukan hak administrator; `chown` mengubah kepemilikan berkas. Keduanya tidak diperlukan pada workspace ini.
5. HTTP 103 adalah respons informasional dan HTTP 503 adalah gangguan layanan. Exit code skrip 0 berarti **klasifikasi berjalan benar**, bukan berarti server HTTP sehat.